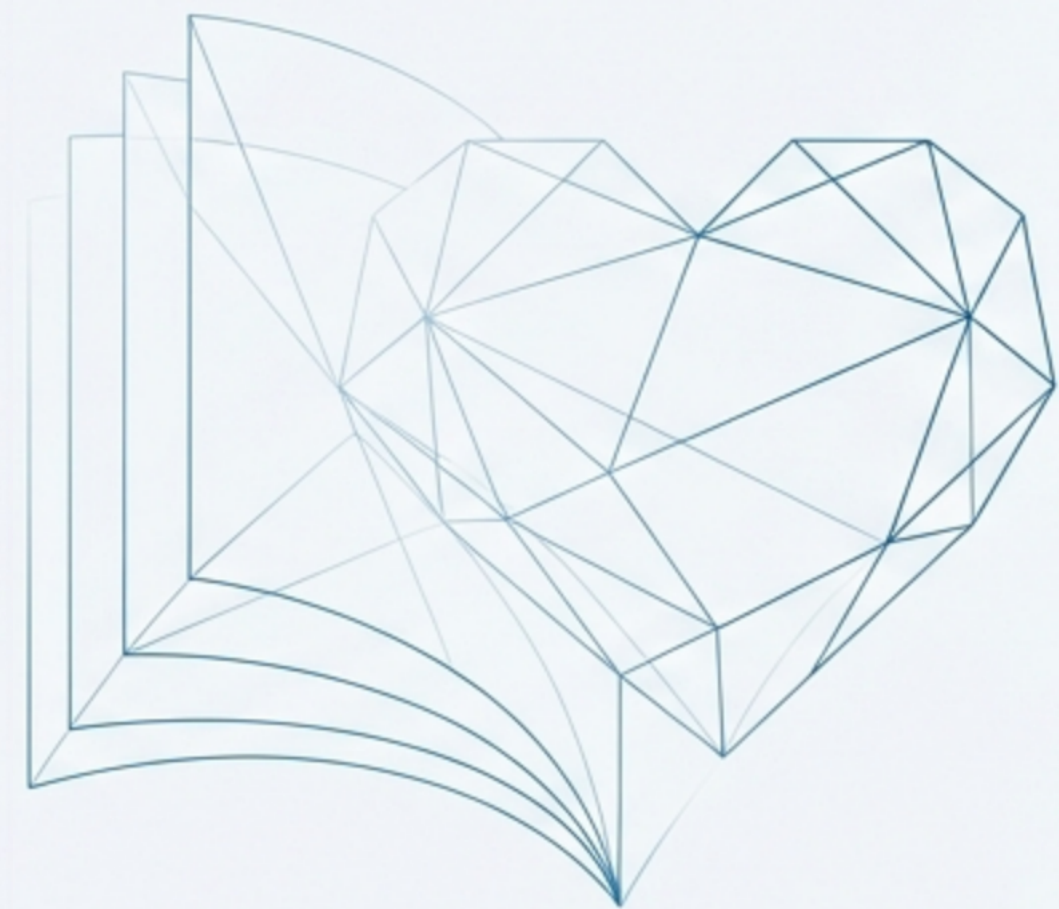


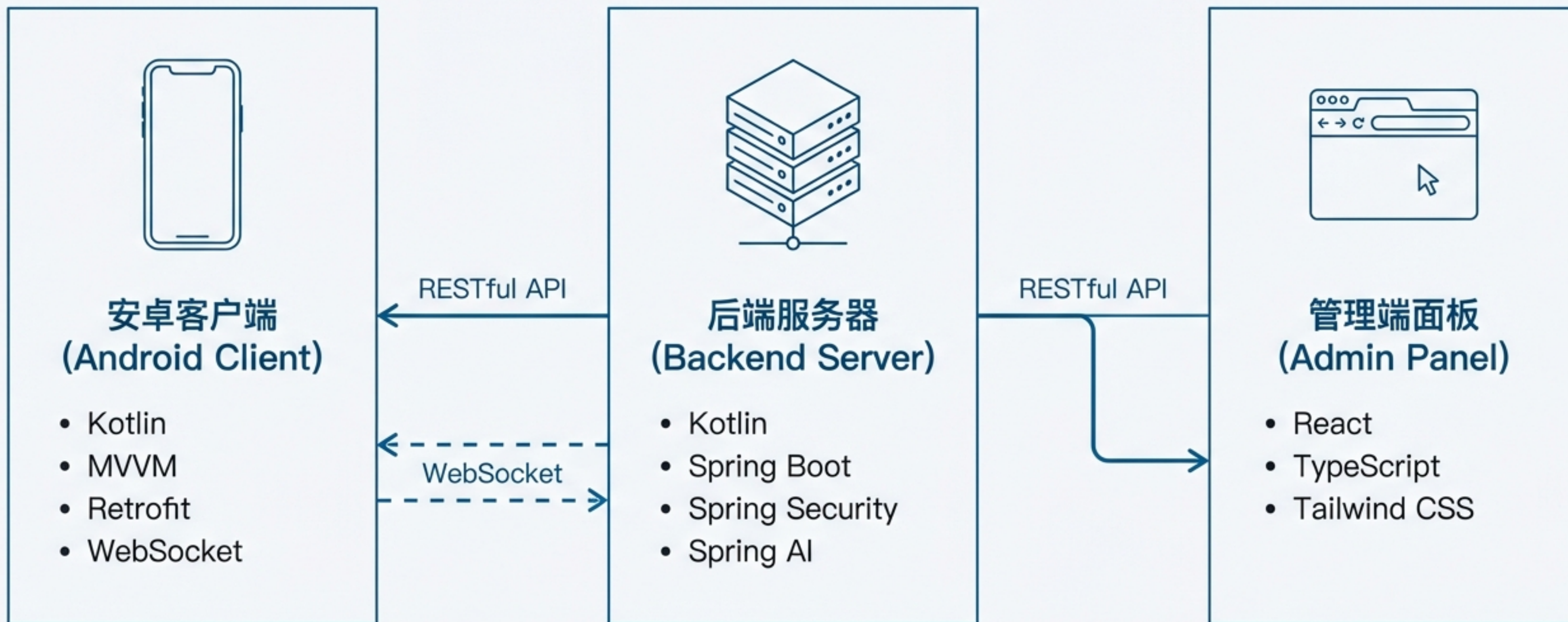
Heartfelt Tutoring

(倾心家教)

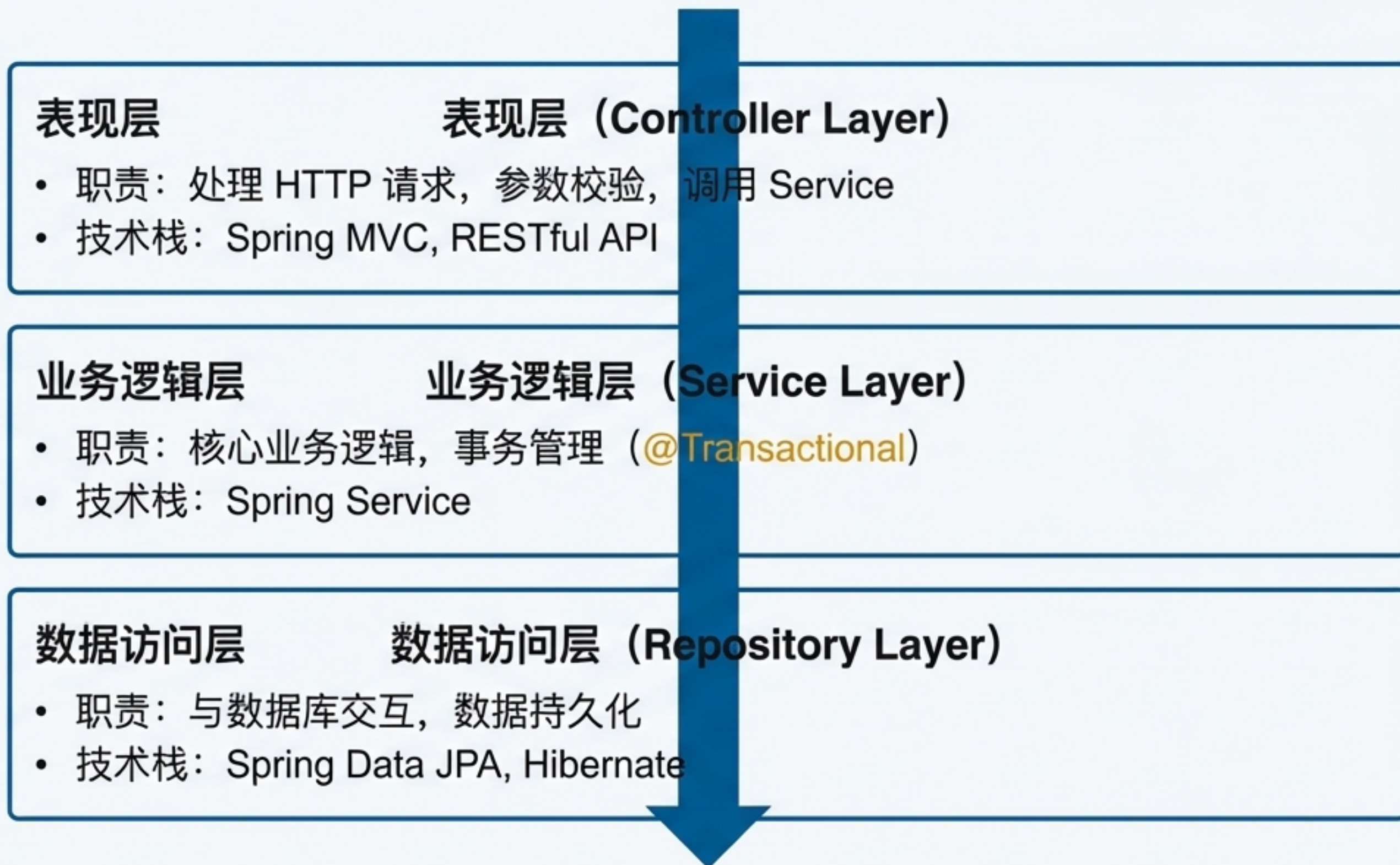
关键技术实现与架构深度解析



系统架构总览：三端一体化平台



后端基石：经典三层架构的可维护性设计



数据模型：从持久化实体（Entity）到接口契约（DTO）的转换

实体类（Entity - POJO）

用途：数据库表映射，JPA 持久化。
关键注解：@Entity, @Table, @Id, @ManyToOne, @Enumerated(EnumType.STRING)

```
@Entity @Table(name = "users")
data class User(
    @Id @GeneratedValue(...)
    var id: Long? = null,
    var username: String,
    @Enumerated(EnumType.STRING)
    var role: Role? = Role.STUDENT,
    ...
)
```

安全映射
(Secure Mapping)



数据传输对象（DTO）

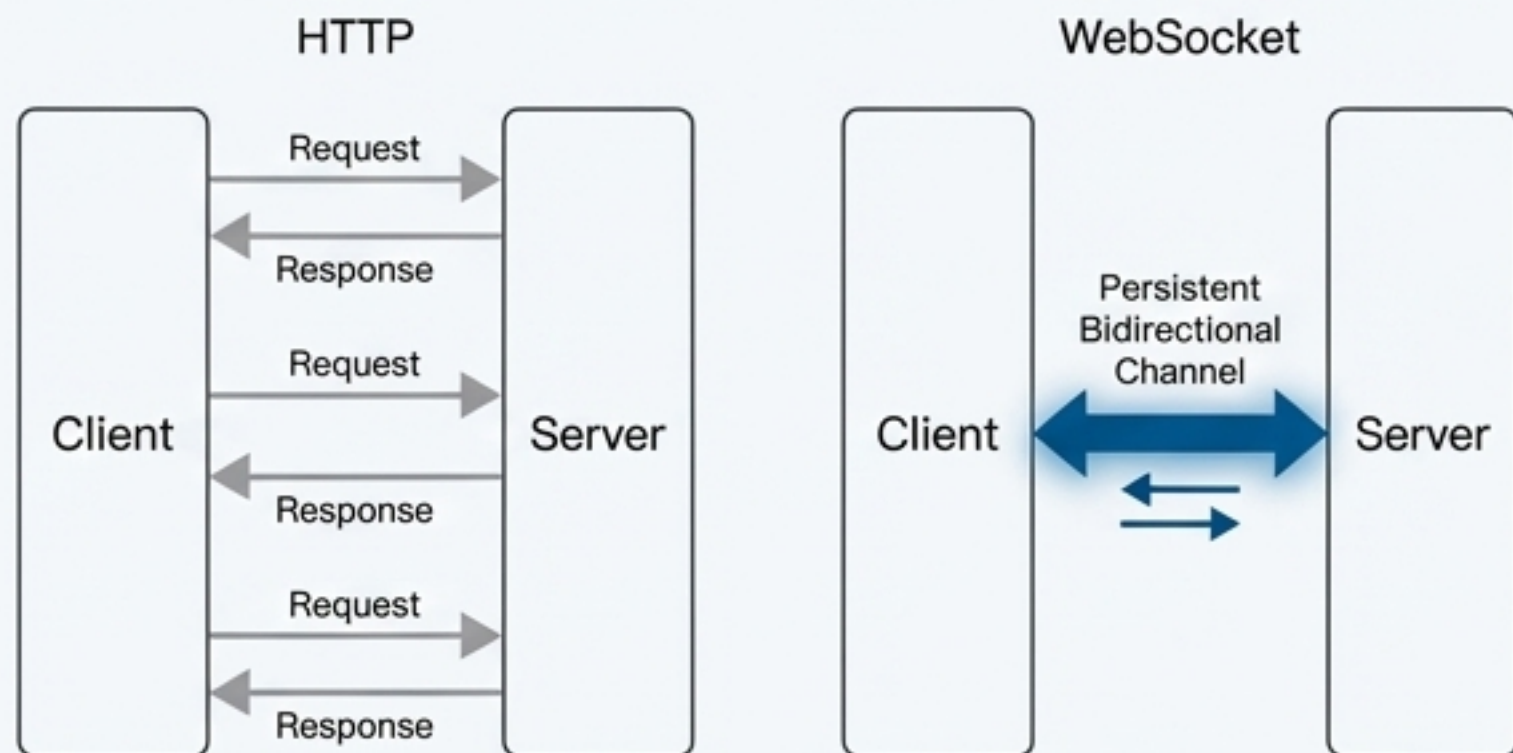
用途：API 数据传输，隐藏内部实现，保障安全。
核心优势：数据封装、避免暴露敏感字段（如密码哈希）、可定制性。

```
data class UserDto(
    var id: Long? = null,
    val username: String? = null,
    val email: String? = null,
    // 密码字段被省略
    ...
)
```


实时交互的核心：WebSocket 连接与 STOMP 协议

WebSocket: The “Connection”

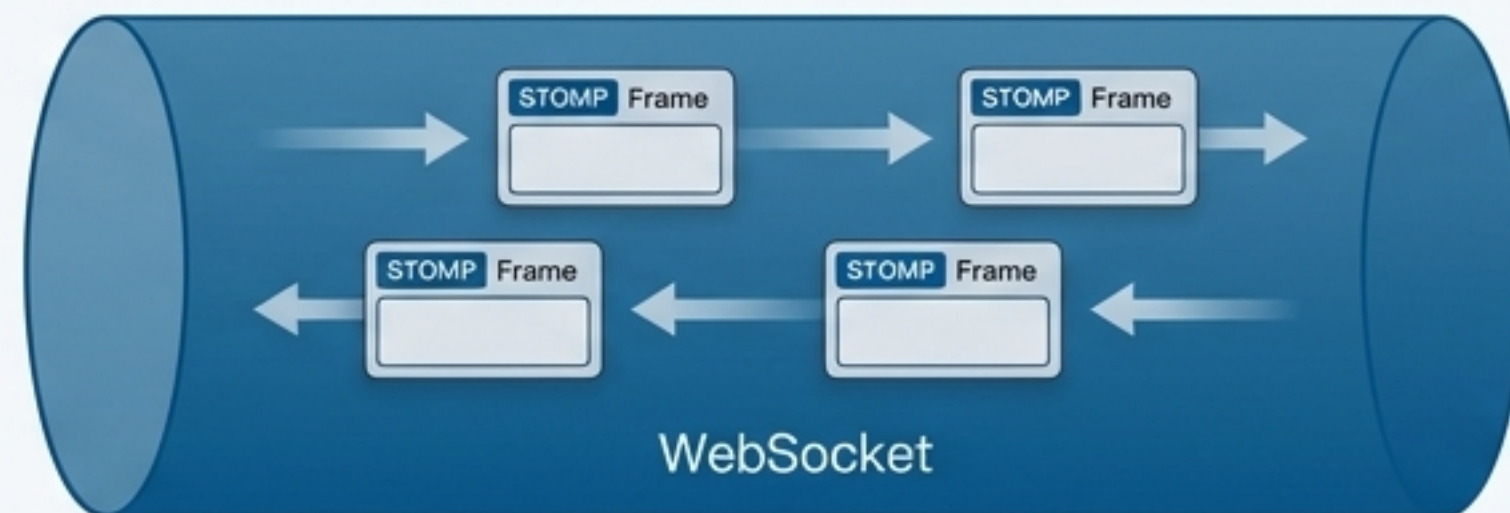
- 提供全双工、低延迟的持久 TCP 连接。
- 解决了 HTTP 轮询的性能瓶颈。



STOMP: The “Protocol”

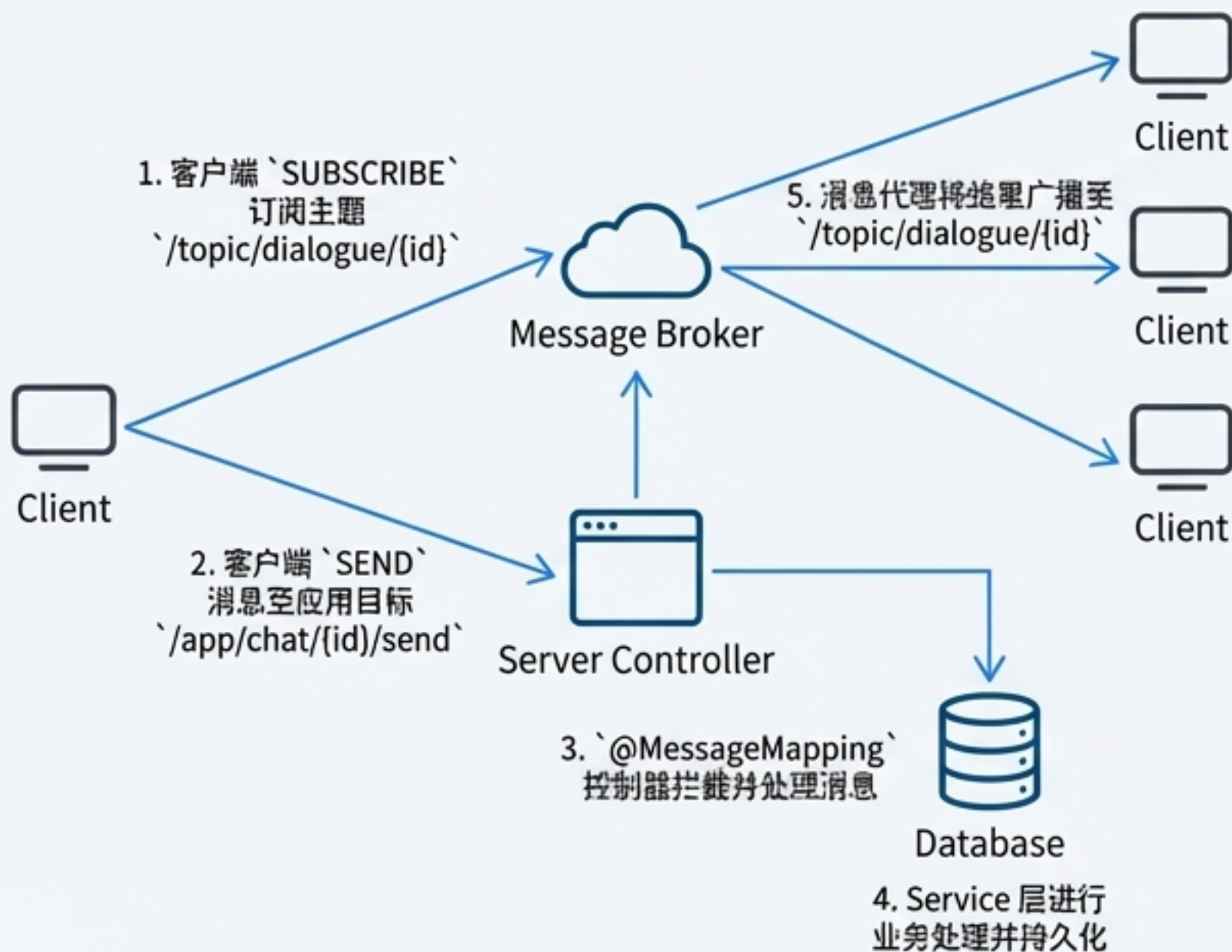
- 运行于 WebSocket 之上，提供更高级的消息语义。
- 关键概念：目的地（Destination）、发布/订阅（Pub/Sub）。

如果 WebSocket 是“管道”，STOMP 就是管道中规范化的“信件格式”，让信息传递更有序。



实时通讯实现：消息代理与控制器路由

消息流转图 (Message Flow Diagram)



核心代码实现 (Core Code Implementation)

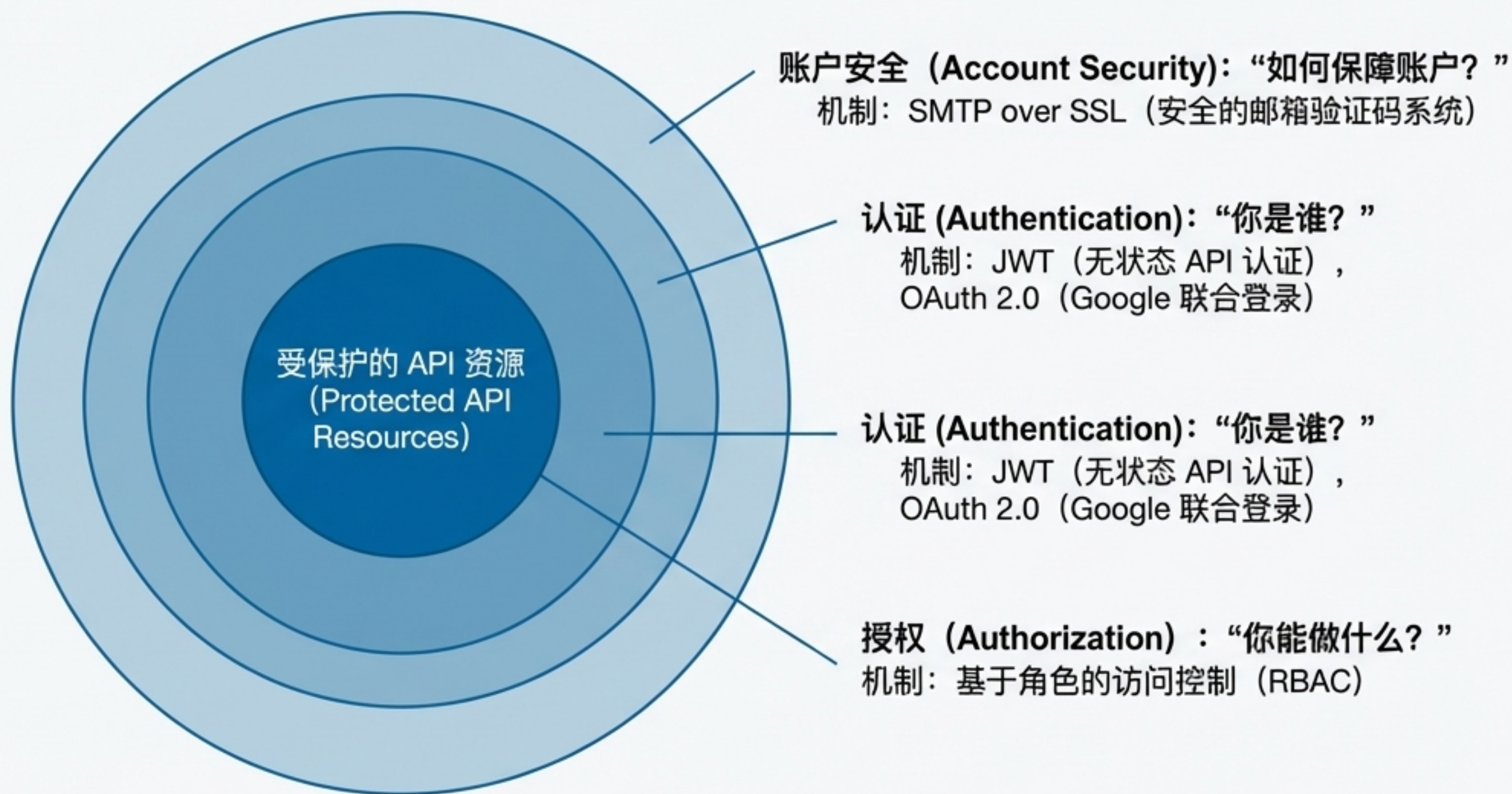
配置 (`WebSocketConfig.kt`)

```
1 // 定义消息代理的主题前缀
2 config.enableSimpleBroker("/topic", "/queue")
3 // 定义应用处理消息的目标前缀
4 config.setApplicationDestinationPrefixes("/app")
```

控制器 (`ChatWebSocketController.kt`)

```
1 @MessageMapping("/chat/{dialogueId}/send")
2 @SendTo("/topic/dialogue/{dialogueId}")
3 fun sendMessage(...) { ... }
```


安全架构：构建纵深防御体系

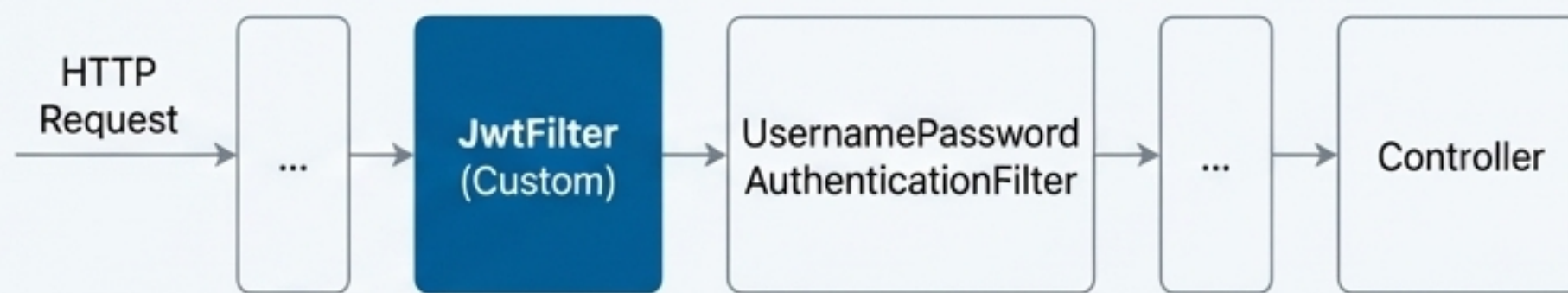


API 的守护者：JWT 与 Spring Security 过滤器链

JWT 工作流程 (JWT Workflow)

1. 用户登录成功，服务器签发包含 `userId` 和 `role` 的 JWT。
2. 客户端在后续请求的 `Authorization` 头中携带 JWT。
3. 服务器端的 `JwtFilter` 拦截请求，验证签名和有效期。
4. 验证通过，构建 `Authentication` 对象并置入 `SecurityContext`。

过滤器链集成 (Filter Chain Integration)



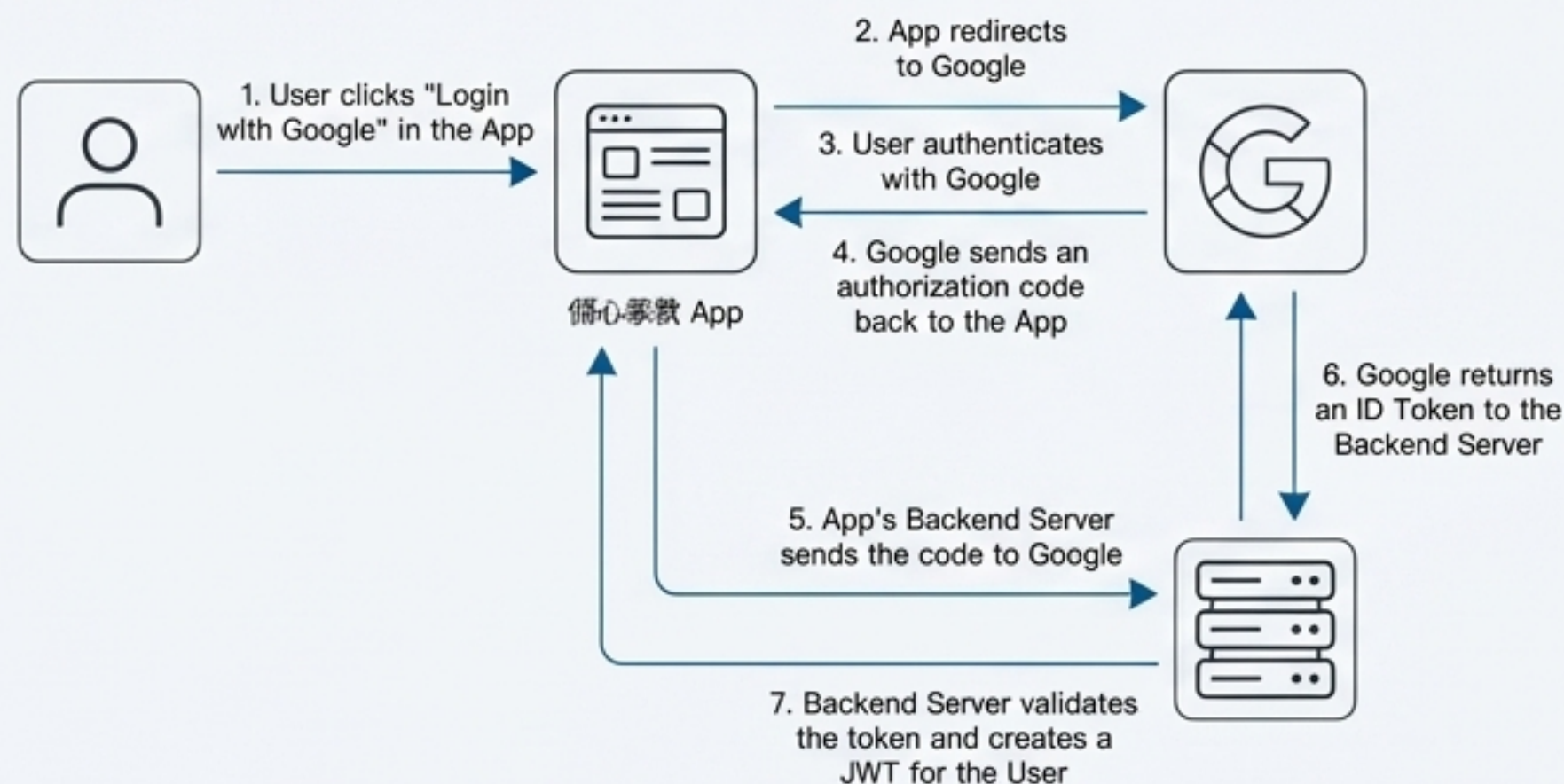
SecurityConfig.kt

```
1 http.addFilterBefore(  
2     jwtFilter,  
3     UsernamePasswordAuthenticationFilter::class.java  
4 )  
5 // 放行认证和WebSocket端点  
6 auth.requestMatchers("/api/auth/**", "/ws/**").permitAll()  
7 // 其他所有请求均需认证  
8 auth.anyRequest().authenticated()
```


提升体验与安全：OAuth 2.0 及邮箱验证

OAuth 2.0 联合认证

核心优势：安全（不接触用户密码）、便捷（一键登录）。



SMTP 邮箱验证

核心目的：防止恶意注册，确认用户身份真实性。

-  安全传输：SMTP over SSL 保证传输加密。
-  时效性控制：验证码 5 分钟有效。
-  防暴力破解：限制尝试次数（5次）。

智能化核心：Spring AI 框架与通义千问模型



框架：Spring AI

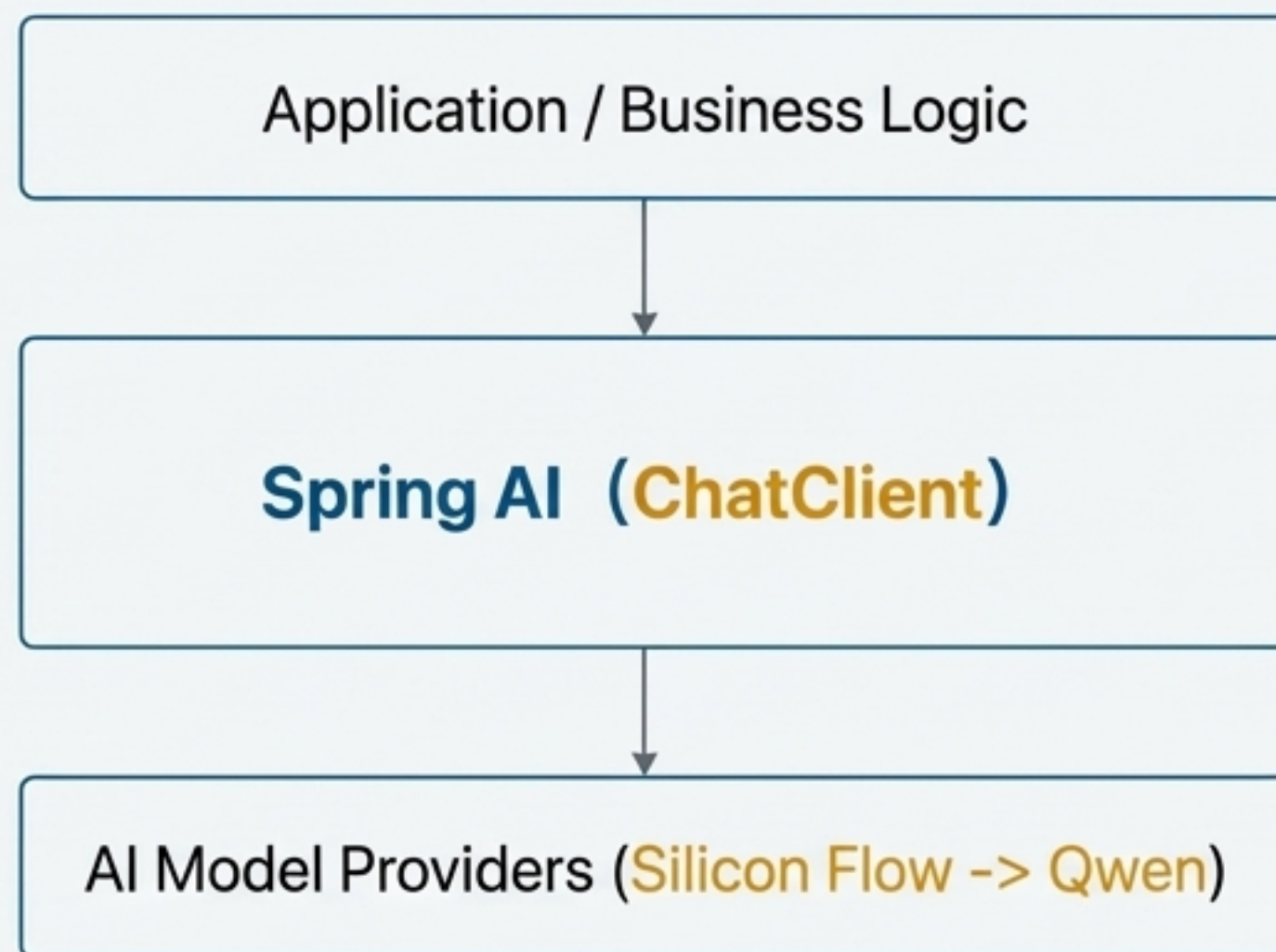
- 为何选择?: Spring 官方支持, 与现有生态无缝集成, 提供统一 **ChatClient** 接口, 解耦具体模型。

模型：Qwen3-30B-A3B-Instruct

- 为何选择?: 强大的中文理解与推理能力, 高度契合中文辅导场景。

应用场景

- 智能问答、学习计划生成、作业辅导。



提升AI交互体验：流式响应与上下文管理

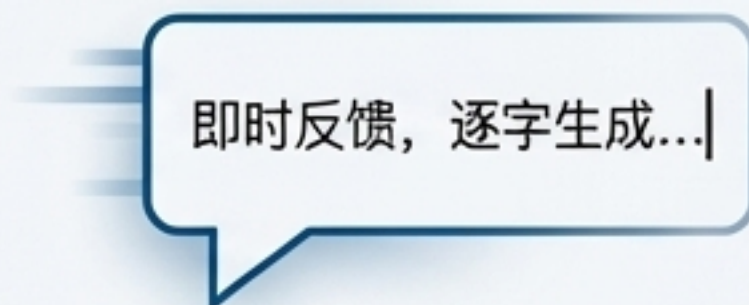
流式响应 (Streaming Response)

传统模式 (Traditional)



等待完整响应...

流式模式 (Streaming)



即时反馈, 逐字生成...

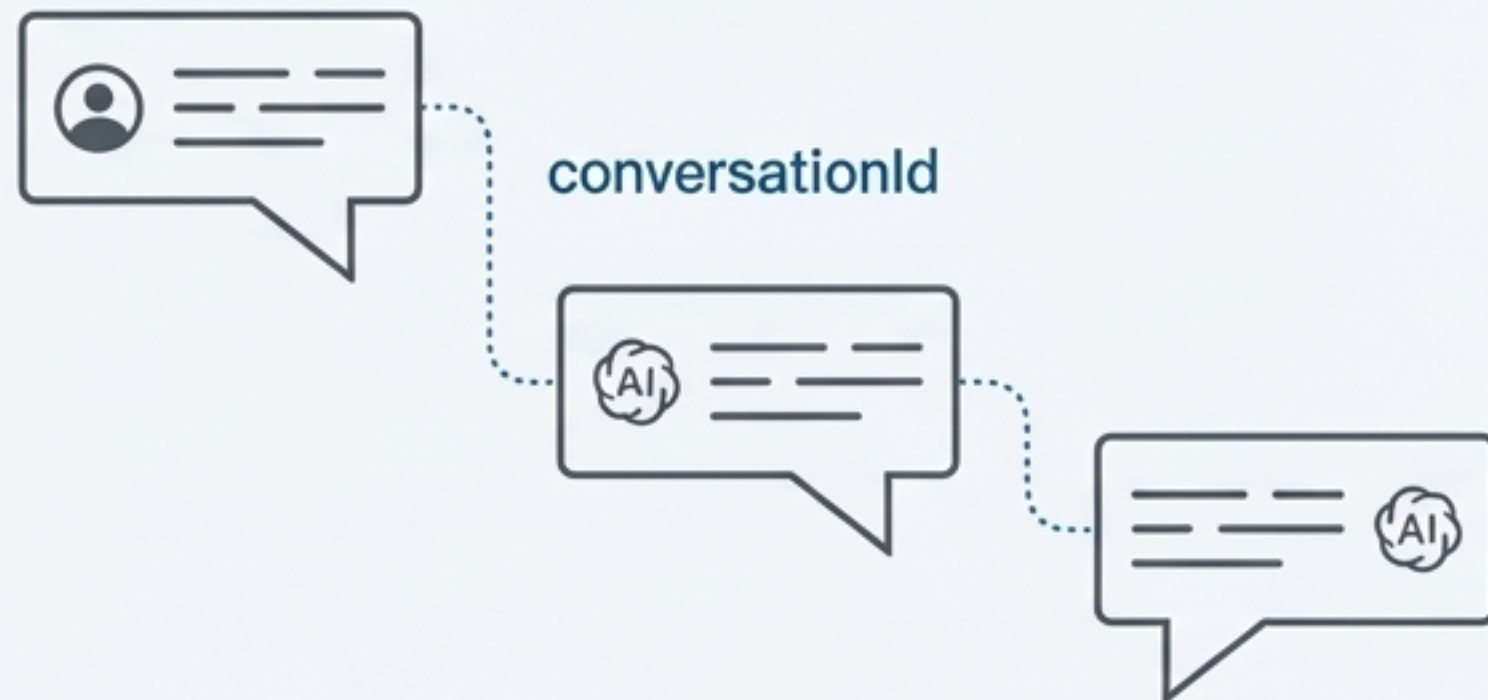
技术实现：后端使用 Reactor 的 `Flux` 类型，通过 SSE (Server-Sent Events) 协议向前端推送数据。

```
// AgentController.kt
@PostMapping("/chat", produces = [MediaType.TEXT_EVENT_STREAM_VALUE])
fun chatWithAIStream(...): Flux<String> {
    return aiService.chatStream(...)
}
```

上下文管理 (Context Management)

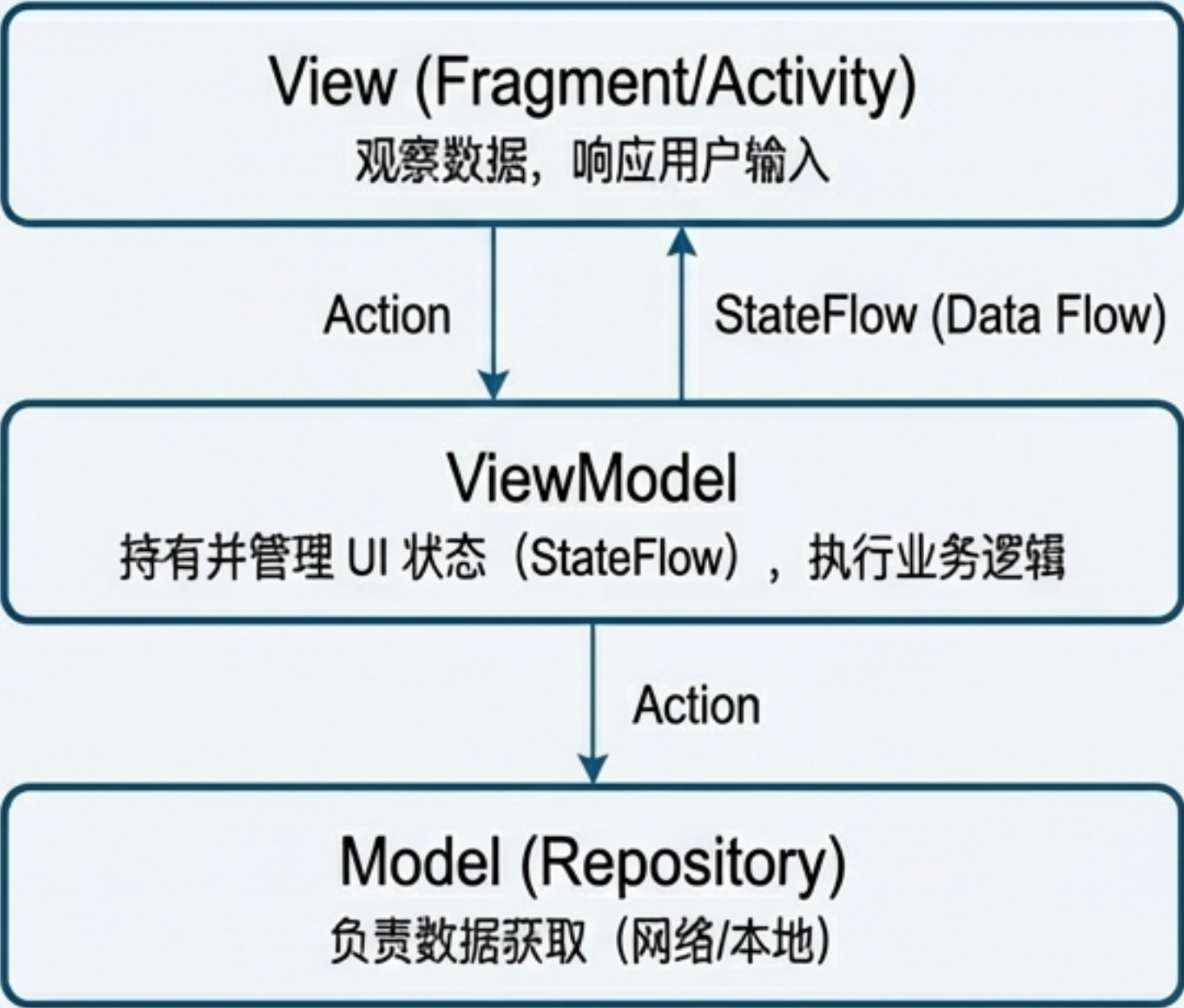
机制：通过 `conversationId` 维护多轮对话历史。

效果：AI 能够理解对话背景，提供连贯、个性化的回答，实现真正的“对话”而非“一问一答”。



安卓客户端：MVVM 架构与协程的健壮性实践

MVVM 架构图 (MVVM Architecture Diagram)



Kotlin 协程并发处理 (Concurrency with Coroutines)

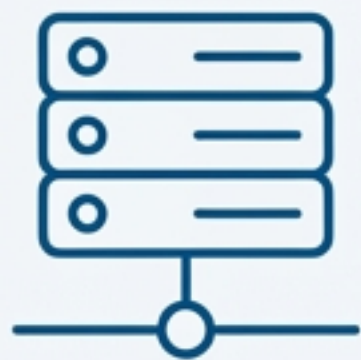
核心优势：避免阻塞主线程，通过 `viewModelScope` 自动管理协程生命周期，有效防止内存泄漏。

```
// PlansViewModel.kt

// 状态流，对View只读
private val _plans = MutableStateFlow<List<Plan>>(emptyList())
val plans: StateFlow<List<Plan>> = _plans.asStateFlow()

fun loadUserPlans() {
    viewModelScope.launch { // 协程与ViewModel生命周期绑定
        val result = repository.getUserPlans(...)
        result.onSuccess { data -> _plans.value = data }
    }
}
```


项目全景技术栈



后端 (Backend)

语言/框架  Kotlin  Spring Boot

数据库  PostgreSQL  JPA/Hibernate

安全  Spring Security  JWT  OAuth 2.0

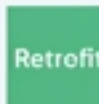
实时  WebSocket  STOMP

AI  Spring AI



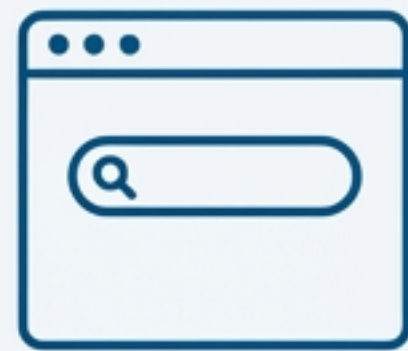
客户端 (Android)

语言/架构  Kotlin, MVVM

网络  Retrofit  OkHttp

UI  Material Design 3

并发  Kotlin Coroutines



管理端 (Admin)

框架  React  TypeScript

样式  Tailwind CSS

项目总结与核心技术成果



后端安全

成功构建了基于 JWT + RBAC + Spring Security 过滤器链的多重安全防护体系。



AI 赋能

集成 Qwen 大模型，并通过流式响应（SSE）和上下文管理显著优化了智能化交互体验。



实时交互

采用 WebSocket/STOMP 协议，构建了低延迟、高可靠的师生及 AI 实时通讯系统。



前端健壮性

客户端严格遵循 MVVM 架构并利用 Kotlin 协程，保证了应用的高性能与高可维护性。

未来工作与展望

功能扩展 (Feature Expansion)

- 完善支付接口对接 (支付宝/微信)
- 增加基于 WebRTC 的视频通话教学功能

性能优化 (Performance Optimization)

- 引入 Redis 缓存层, 提高热点数据查询速度

AI 增强 (AI Enhancement)

- 集成更多专用工具 (如文件处理、代码解释)

我的报告到此结束, 欢迎各位老师和同学提问交流。